A Classical-AI Reasoning Agent for Logical Entailment

Palak Jat • Roll No: M26MAC006 • AI Assignment • Notebook: `reasoning_agent.ipynb`

> **Overview:** This system evaluates whether a given query logically follows from a knowledge base of facts and rules. The pipeline converts heterogeneous inputs—first-order logic statements in Unicode, ASCII, or Prolog syntax, alongside templated English sentences—into a unified Abstract Syntax Tree (AST). Reasoning is handled by three complementary classical inference systems: semi-naive forward chaining with unification and stratified negation-as-failure, resolution refutation with set-of-support and best-first clause ordering, and structural causal models with exhaustive search abduction. The system contains zero neural parameters and uses no generative AI.

SYSTEM PERFORMANCE SUMMARY

Dataset	Target Semantics	Queries	Accuracy %	Total Time	Speed
RuleTaker (Full test split)	Closed World (NAF)	151,911	99.97%	71.7 s	0.47 ms/q
FOLIO (Train + Val, Gold FOL)	Open World (FOL)	1,204	82.72%	37.2 s	30.90 ms/q
CounterBench (Alpha + Backdoor)	Causal Interventions	1,200	95.00%	0.5 s	0.42 ms/q
All Queries (Micro / Macro Avg)	**Combined Semantics**	**154,315**	**99.80% / 92.57%**	**109.4 s**	**0.71 ms/q**

Table 1: Overall accuracy and runtime performance on single CPU core (Python 3.11). Runtimes include complete input parsing, normalisation, and inference steps.

1. KNOWLEDGE REPRESENTATION

All input streams are parsed into an Abstract Syntax Tree (AST) representing first-order logic formulas. Terms inside the AST are formatted either as variables (indicated by a ? prefix, e.g., `?x`), constants (e.g., bob), or compound function terms. Formulas are structured as nested tuples covering atomic predicates, negations, conjunctions, disjunctions, implications, equivalences, exclusive ORs, and quantifiers: `('atom', pred, args)`, `('not', f)`, `('and', ...)`, `('or', ...)`, `('imp', a, b)`, `('iff', a, b)`, `('xor', a, b)`, `('forall', var, f)`, and `('exists', var, f)`.

Each engine derives its required working structure directly from this primary AST:

- **Fact & Rule Database (Engine A):** Stores simple true/false statements and basic "if-then" rules. Both the conditions and conclusions can use negative statements.
- **Clause Form (Engine B):** Standardises complex logic statements into clean sets of combined conditions. It removes complex arrows, assigns distinct names to variables, and turns existential statements into concrete representations.
- **Causal Formulas (Engine C):** Represents cause-and-effect rules (for instance, turning "A and B together cause not C" into a basic logical rule) and keeps track of basic starting probabilities for unknown factors.

Reading Inputs & Cleaning Names: A custom parser reads logic statements written in standard math

2. CLASSICAL REASONING ALGORITHMS

Engine A: Forward Chaining with Stratified Negation-as-Failure

Engine A handles rule bases under Closed-World Assumption (CWA) semantics. Ground facts and rules are matched against known assertions using one-way substitution. Semi-naive iteration fires matching rule instances until no new conclusions can be added. Explicit negative conclusions (like `¬P(c)`) add negative facts directly to the database.

Body literals containing `not P(t)` are evaluated under negation-as-failure (NAF). To keep logical evaluation consistent, the system builds a predicate dependency graph and groups predicates into strata. Predicates are evaluated stratum by stratum, so a negative condition `not P` is only queried after all rules that could produce P have completed. Unbound variables inside NAF literals range across all known domain constants (Herbrand universe).

Engine B: First-Order Resolution Refutation

Engine B handles Open-World Assumption (OWA) reasoning. To prove that a knowledge base entails query *q*, the engine negates *q* and attempts to derive the empty clause (□) from KB ∪ {¬q}.

The negated query forms the *set of support*. The main loop uses an Otter-style search, pulling the lightest clause from a priority queue sorted by symbol count. Selected clauses are resolved against stored clauses using most-general unification with an occurs check. Factoring is applied immediately. Tautologies, duplicate clauses, clauses exceeding 6 literals, and clauses deeper than 3 nested function levels are discarded. Equality is handled using reflexivity, symmetry, transitivity, and substitution axioms, along with the unique-names assumption for constants that are not equated in premises.

Engine C: Structural Causal Models (Counterfactuals)

Engine C processes counterfactual queries (such as "Would Y occur if not X instead of X?"). Reasoning follows Judea Pearl's abduction-action-prediction steps:

1. **Abduction:** Enumerate all binary assignments to root exogenous variables using depth-first search, filtering for assignments that match observed facts.
2. **Action:** Apply the counterfactual intervention *do(·)* by replacing selected structural equations with fixed assignment values.
3. **Prediction:** Forward-propagate state values through structural equations in topological order (using up to three sweeps to settle cyclic structures). The query returns `True` if P(Y) > 0.5.

ENGINE SELECTION ROUTING

> Queries containing counterfactual syntax ("would ... occur if") route to **Engine C**. Knowledge bases structured as rule bases with ground literal queries and unary/binary equality-free predicates route to **Engine A**. All general first-order logic problems route to **Engine B**.

3. EXPERIMENTAL RESULTS ON BENCHMARK DATASETS

Evaluation measures accuracy on official benchmark test sets without using any training examples, memory tables, or neural models. Labels map directly to binary flags: RuleTaker `entailment` → `True`; FOLIO `True` → `True` (and `False` / `Uncertain` → `False`); CounterBench `yes` → `True`.

RuleTaker Accuracy by Depth

Subset	Queries	Accuracy %
NatLang	7,793	100.00%
depth-0	17,347	100.00%
depth-1	17,788	100.00%
depth-2	17,741	99.99%
depth-3	17,982	99.96%
depth-3ext	23,222	99.97%
depth-3ext-NatLang	31,017	99.97%
depth-5	19,021	99.92%

Table 2: RuleTaker test performance broken down by proof depth.

CounterBench & FOLIO Splits

Subset	Queries	Accuracy %
CounterBench / backdoor	200	90.00%
CounterBench / basic	250	100.00%
CounterBench / conditional	250	92.00%
CounterBench / joint	250	96.00%
CounterBench / nested	250	96.00%
FOLIO / train	1,001	81.82%
FOLIO / validation	203	87.19%

Table 3: Accuracy breakdown across reasoning types in CounterBench and FOLIO splits.

ABLATION EXPERIMENTS

Experiment 1: Input Syntax & CWA vs OWA (RuleTaker)

Configuration	Queries	Accuracy %	Time (s)
RuleTaker as logic (Unicode symbols)	5,000	100.00%	17.8 s
RuleTaker as logic (ASCII operators)	5,000	100.00%	13.6 s
RuleTaker as logic (Word operators)	5,000	100.00%	5.8 s
RuleTaker, Closed World (Forward Chaining + NAF)	1,500	100.00%	0.8 s
RuleTaker, Open World (Resolution Refutation)	1,500	74.27%	3.0 s

Table 4: Evaluation of input syntax conversion and reasoning mode on RuleTaker queries.

Converting English sentences into explicit logic syntaxes retains 100% accuracy, showing that the logic parser loses no information compared to the English grammar parser. However, forcing an Open World resolution engine onto RuleTaker drops accuracy to 74.27% because resolution cannot prove `¬P(c)` when `P(c)` is simply un-derivable, confirming that RuleTaker requires Closed-World NAF semantics.

4. DETAILED ABLATIONS & ERROR ANALYSIS

Experiment 2: FOLIO Ablation Studies

FOLIO Configuration	Accuracy %	Inference Time (s)
Resolution only, raw symbols, no UNA	79.82%	34.5 s
+ Symbol normalisation	82.81%	33.4 s
+ Unique-names assumption (UNA)	82.89%	33.6 s
+ CWA engine for rule-shaped KBs (Final system)	82.72%	33.0 s
Final system, prover search limit = 0.05 s	82.64%	13.6 s
Final system, prover search limit = 0.50 s	82.81%	96.4 s

Table 5: Impact of individual modules and time limits on FOLIO accuracy.

Symbol normalisation delivers the largest gain (+2.99%), fixing minor spelling mismatches in the annotations (such as `Residentof` vs `ResidentOf` or `brianwinter` vs `brian Winter`). The unique-names assumption adds minor gains on counting problems. Changing the resolution timeout from 0.05s to 0.50s provides minimal accuracy gain while significantly increasing runtime, showing that valid proofs are found quickly.

FOLIO Confusion Matrix

Gold Label	Predicted True	Predicted False
True (297 entailed)	297	163
False (351 non-entailed)	29	322
Uncertain (393 unproven)	16	377

Table 6: Confusion matrix comparing gold labels with predictions.

Experiment 3: CounterBench Variants

Testing adjustments to the counterfactual algorithm:

- Uniform priors (p=0.5) + evidence relaxation (Final): **95.00%**
- Exogenous causes default to 0: **90.00%**
- Exogenous causes default to 1: **95.00%**
- Without evidence relaxation: **94.58%**
- Conditioning on factual treatment value: **95.00%**
- Causal story as propositional logic: **95.00%**

ANALYSIS OF ERROR MODES

The remaining 5% error margin in CounterBench stems from template patterns whose gold labels contradict their stated structural equations. In FOLIO, false positives are rare because resolution is sound. Most FOLIO errors occur when gold-True conclusions depend on unstated real-world knowledge or contain malformed logic formulas that cannot be derived strictly from the premises.

5. EXECUTION TRACES & VERIFICATION

Forward Chaining Trace (Closed World - RuleTaker)

```
CONTEXT: Anne is kind. Kind things are blue. If something is blue and not red then it is
```

Resolution Refutation Trace (Open World - FOLIO)

```
CONTEXT: ∀x (DrinkRegularly(x, coffee) → IsDependentOn(x, caffeine)) ∀x (DrinkRegularly(x
```

Structural Causal Model Trace (CounterBench)

```
CONTEXT: We know that Fizo causes Blorn, Blorn causes Plim, Plim causes Quaz, and Quaz ca
```

SYSTEM COMPLEXITY & LIMITS

Forward chaining processes RuleTaker queries in ~0.47 ms per query. Resolution averages ~30.9 ms on FOLIO, where runtime is dominated by budget-limited searches that fail to find a proof. Causal model propagation completes in ~0.42 ms per query. Across all 154,315 public queries, total runtime is 109.4 seconds on a single CPU core. Main limitations stem from input errors: because entailment is computed purely symbolically, malformed input formulas that symbol normalisation cannot repair will default to underivable (False).